

Question-1:

You are given a triangle of integers represented as a 2D array `triangle`.

Starting from the **top of the triangle**, you must move down one row at a time until you reach the **bottom row**.

From a position at index `i` in the current row, you can move to either:

- index `j` in the next row, or
- index `j + 1` in the next row.

Your task is to find the **minimum possible sum of all values along a path from the top to the bottom**.

Example 1

Input:

```
triangle = [  
    [2],  
    [3, 4],  
    [6, 5, 7],  
    [4, 1, 8, 3]  
]
```

One possible minimum path is:

$2 \rightarrow 3 \rightarrow 5 \rightarrow 1$

Its sum is:

$2 + 3 + 5 + 1 = 11$

Output:

11

Example 2

Input:

```
triangle = [  
    [-10]  
]
```

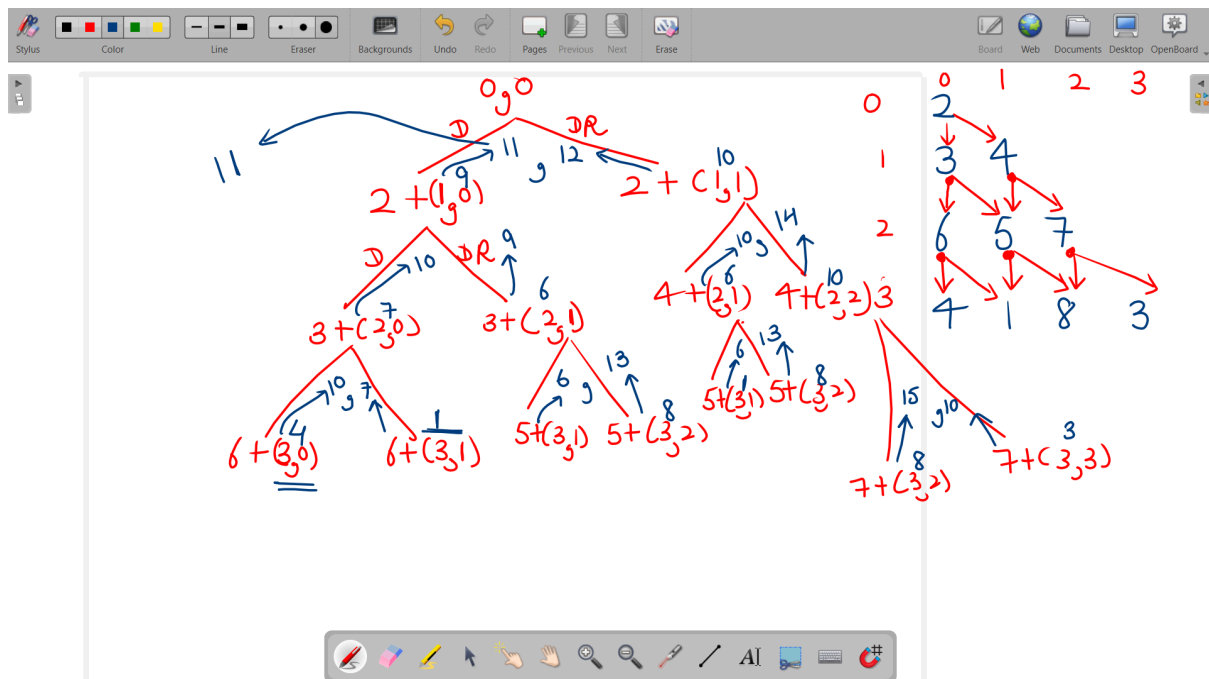
There is only one possible path.

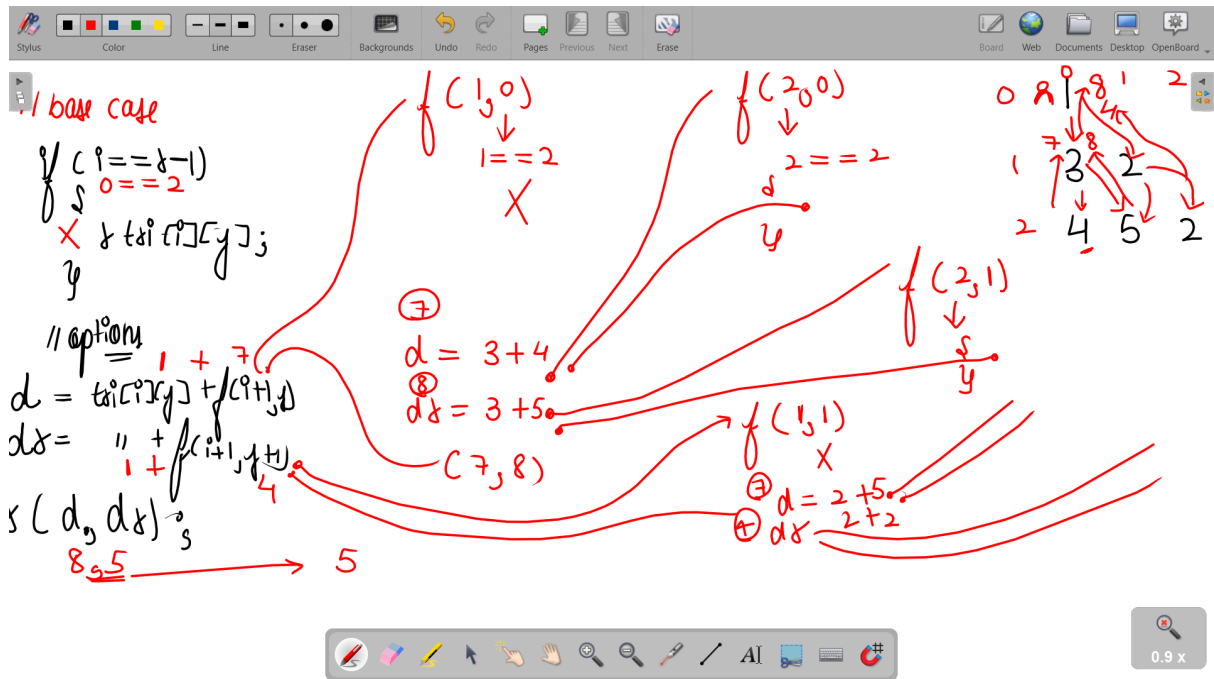
Output:

-10

Constraints

- $1 \leq \text{triangle.length} \leq 200$
- $\text{triangle}[0].\text{length} == 1$
- $\text{triangle}[i].\text{length} == \text{triangle}[i - 1].\text{length} + 1$
- $-10^4 \leq \text{triangle}[i][j] \leq 10^4$





Question-2:

You are given an integer N representing the size of a square chessboard of dimensions $N \times N$.

A Knight is placed at the top-left corner $(0, 0)$ of the board. Your task is to find a valid Knight's Tour, in which the Knight visits every cell exactly once.

A Knight can move in an L-shaped pattern: two cells in one direction followed by one cell perpendicular to that direction.

For example, if the Knight is currently at $(2, 2)$, its possible moves are:

$(4, 3), (4, 1),$

$(0, 3), (0, 1),$

$(3, 4), (3, 0),$

$(1, 4), (1, 0)$

The Knight must remain within the boundaries of the chessboard and cannot visit the same cell more than once.

Task

Return a matrix representing the order in which the cells are visited.

The starting cell $(0, 0)$ must contain 0 , the next visited cell must contain 1 , the following cell 2 , and so on until all $N \times N$ cells have been visited.

If a valid Knight's Tour exists, return any one valid tour.

If no valid tour is possible, return an empty matrix.

Examples

Example 1

Input:

$N = 5$

Output:

```
[ 0, 11, 2, 17, 20],  
[ 3, 16, 19, 12, 7],  
[10, 1, 6, 21, 18],  
[15, 4, 23, 8, 13],  
[24, 9, 14, 5, 22]  
]
```

Explanation:

The Knight starts at $(0, 0)$, represented by 0.

It then moves to the cell containing 1, followed by 2, 3, and so on.

The final cell visited contains 24, meaning all 25 cells of the 5×5 board have been visited exactly once.

Since every consecutive number represents a valid Knight move, this is a valid Knight's Tour.

Example 2

Input:

$N = 4$

Output:

[]

Explanation:

A valid Knight's Tour cannot be completed on a 4×4 chessboard when starting from $(0, 0)$.

Therefore, the function should return an empty matrix.

Example 3

Input:

$N = 1$

Output:

```
[  
  [0]  
]
```

Explanation:

The board contains only one cell. Since the Knight starts at $(0, 0)$, it has already visited the entire board.

Therefore, the tour is simply:

0

Constraints

$1 \leq N \leq 6$

Important Conditions

- The Knight must start from $(0, 0)$.
- Every cell must be visited exactly once.
- A Knight can only make valid Knight moves.
- The Knight cannot move outside the board.

- The returned matrix must contain every number from 0 to $N \times N - 1$ exactly once.
- If no valid tour exists, return an empty matrix.
- If multiple solutions exist, any valid solution is acceptable.

Solution:

```
class Solution {
public:
    int dx[8]={-1,-1,1,1,-2,-2,2,2};
    int dy[8]={2,-2,2,-2,-1,1,-1,1};

    bool fillMatrix(int i,int j,int move,int n,vector<vector<int>>&result){
        //edge case
        if(i<0||j<0||i>=n||j>=n||result[i][j]!=-1){
            return false;
        }

        //base case
        if(move==n*n-1){
            result[i][j]=move;
            return true;
        }

        //main logic
        result[i][j]=move;

        for(int t=0;t<8;t++){
```

```

        int newi=i+dx[t];

        int newj=j+dy[t];

        if(fillMatrix(newi,newj,move+1,n,result)==true) return true;
    }

    result[i][j]=-1;

    return false;

}

vector<vector<int>> knightTour(int n) {
    vector<vector<int>>result(n,vector<int>(n,-1));

    int i=0;

    int j=0;

    int move=0;

    if(fillMatrix(i,j,move,n,result)==true){
        return result;
    }

    else{
        return {};
    }

}

};

```

